EXPERIMENT NO. 1

AIM:

To implement the Insertion Sort algorithm in the C programming language to sort an array of integers in ascending order.

THEORY:

Insertion sort is a simple and efficient sorting algorithm that works similarly to the way we sort playing cards in our hands. It is an in-place sorting algorithm, which means it sorts the elements within the original array without requiring extra memory or auxiliary data structures.

How it Works:
1. The algorithm virtually splits the array into a sorted part and an unsorted part.
2. Elements from the unsorted part are picked one by one and inserted into their correct position in the sorted part.
3. This is done by shifting all greater elements to the right to make space for the picked element.

Insertion Sort Algorithm:

```
for j = 2 to A.length
    key = A[j]
    // Insert A[j] into the sorted sequence A[1 .. j-1]
    i = j - 1
    while i > 0 and A[i] > key
        A[i+1] = A[i]
        i = i - 1
    A[i+1] = key
```

WORKING EXAMPLE (TRACING STEPS):

Consider an unsorted array: [6, 2, 11, 7, 5]. Below is the step-by-step trace. (Note: Green represents the sorted sub-array, and Orange represents the active key element being inserted).

Step 1: Initial state. The first element [6] is trivially sorted. The second element (Key = 2) is selected to find its position.

6	**2**	11	7	5

Step 2: Since 2 < 6, 6 is shifted right and 2 is inserted at the front. Now, [2, 6] is the sorted sub-array. The next element (Key = 11) is selected.

2	6	**11**	7	5

Step 3: Since 11 > 6, no shifting occurs. The sorted sub-array expands to [2, 6, 11]. The next element (Key = 7) is selected.

2	6	11	**7**	5

Step 4: 7 is compared with the sorted sub-array. Since 7 < 11 but 7 > 6, 11 is shifted right, and 7 is inserted. Next, the final element (Key = 5) is selected.

2	6	7	11	**5**

Step 5 (Final Sorted Array): 5 is compared with the elements. 11, 7, and 6 are shifted right, and 5 is placed right after 2. The array is fully sorted!

2	5	6	7	11

Complexity Analysis:

Case	Time Complexity
Best Case (Already Sorted)	O(n)
Average Case	$O(n^2)$
Worst Case (Reverse Sorted)	$O(n^2)$

Space Complexity: O(1) auxiliary space.

PROGRAM:

```
#include <stdio.h>
#include <string.h>

void insertionSort(int arr[], int n) {
```

```
    int i, key, j;
    for (i = 1; i < n; i++) {
        key = arr[i];
        j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}

int main() {
    char input[100];
    int arr[100], n;

    printf("Enter a number (e.g. 3421): ");
    scanf("%s", input);  // take input as a string

    n = strlen(input);   // number of digits
    for (int i = 0; i < n; i++) {
        arr[i] = input[i] - '0'; // convert char to int
    }

    insertionSort(arr, n);

    printf("Sorted digits:\n");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    return 0;
}
```

CONCLUSION:

Hence, the Insertion Sort algorithm was successfully implemented and verified in C. The step-by-step graphic trace highlights how a sorted subarray is built incrementally. We verified that the algorithm works with a best-case time complexity of O(n) and uses a constant O(1) space.